

Produção, transmissão e atendimento da demanda de energia por programação linear

Autor: Iago Cardoso Bariuka

Departamento de Informática, Universidade Federal do Paraná (UFPR) — CII238/INFO7070, Otimização

Resumo. Este artigo apresenta a modelagem e a implementação de um problema de otimização de uma rede de energia formada por hidrelétricas, centrais elétricas e arcos de transmissão. O objetivo é minimizar o custo de produção e transmissão, respeitando a vazão disponível no rio, as capacidades das hidrelétricas e dos arcos e as demandas das centrais. O modelo é um programa linear, e o programa em C desenvolvido lê uma instância do problema e gera, na saída padrão, um arquivo compatível com o formato de entrada do resolvidor *lp_solve*.

Palavras-chave: programação linear; fluxo em rede; hidrelétricas; transmissão de energia; *lp_solve*.

1. Introdução

O planejamento de uma rede elétrica precisa decidir quanto cada hidrelétrica deve produzir e por quais conexões a energia deve ser transmitida. Uma decisão inadequada pode exceder a capacidade de produção ou de transmissão, deixar centrais sem atendimento ou usar uma combinação mais cara do que o necessário. O problema deste trabalho consiste em encontrar uma distribuição de produção e de fluxo que atenda a todas as demandas com o menor custo possível.

Consideram-se h hidrelétricas instaladas em um mesmo rio e l centrais elétricas, ligadas por uma rede de transmissão direcionada. Cada hidrelétrica possui uma capacidade máxima de produção, uma eficiência que converte vazão de água em energia e um custo por unidade de vazão utilizada; sua vazão individual está, além disso, limitada pela vazão do rio, que é compartilhada por todas elas. Cada arco da rede possui uma capacidade de transmissão e um custo por unidade de energia transportada. Cada central possui uma demanda que deve ser atendida pela energia que chega até ela.

Como o custo de produção e o custo de transmissão são proporcionais, respectivamente, à vazão usada e à energia transmitida, o problema é linear e pode ser resolvido de forma exata por programação linear, sem que seja necessário implementar o próprio método simplex. Coube à implementação apenas ler a descrição de uma instância e gerar, na saída padrão, o programa linear correspondente no formato de entrada do *lp_solve* [1], deixando a resolução propriamente dita a cargo desse resolvidor.

2. Modelagem

2.1 Conjuntos e parâmetros

Sejam:

- $H = \{1, \dots, h\}$: conjunto de hidrelétricas;
- $L = \{1, \dots, l\}$: conjunto de centrais;

- A : conjunto de arcos da rede de transmissão, cada um com origem, destino, capacidade w_k e custo de transmissão c_k ;
- M_i : capacidade máxima de produção da hidrelétrica i ;
- F_i : eficiência da hidrelétrica i ;
- C_i : custo por unidade de vazão da hidrelétrica i ;
- D_k : demanda da central k ;
- R : vazão máxima disponível no rio, compartilhada por todas as hidrelétricas;

Os $h + l$ vértices da rede são numerados de 1 a $h + l$: as hidrelétricas ocupam os índices 1, ..., h (na ordem em que aparecem na entrada) e as centrais ocupam os índices $h + 1$, ..., $h + l$ — isto é, a central k corresponde ao vértice global $h + k$. É essa numeração global que o programa usa internamente para o campo origem de cada arco. Já o campo destino de um arco é lido diretamente da entrada e representa o índice da central de destino (1 a l), não o índice global do vértice, convenção adotada no próprio enunciado e respeitada em toda a implementação.

2.2 Variáveis de decisão

- $V_i \geq 0$: vazão utilizada pela hidrelétrica i ;
- $e_k \geq 0$: energia transmitida pelo arco k ;

A produção da hidrelétrica i é dada por $F_i \times V_i$. Por isso a produção não precisa ser uma variável independente do modelo: toda vez que ela aparece, é substituída diretamente por $F_i \times V_i$, e a única variável de decisão do lado da produção é a vazão.

2.3 Função objetivo

$$\text{Minimizar: } Z = \sum_{i=1}^h C_i \times V_i + \sum_{k=1}^n c_k \times e_k$$

Onde n é o número total de arcos da rede. O primeiro termo é o custo total de produção, custo por unidade de vazão multiplicado pela vazão de cada hidrelétrica; o segundo é o custo total de transmissão, custo de cada arco multiplicado pela energia nele transmitida.

2.4 Restrições

2.4.1 Capacidade de produção

Para cada hidrelétrica i ($1 \leq i \leq h$): $F_i \times V_i \leq M_i$

A energia efetivamente produzida por uma hidrelétrica não pode ultrapassar sua capacidade máxima.

2.4.2 Vazão do rio

Para cada hidrelétrica i ($1 \leq i \leq h$): $V_i \leq R$

O enunciado especifica que a vazão de cada hidrelétrica, individualmente, está limitada pela vazão do rio.

2.4.3 Capacidade dos arcos

Para cada arco k ($1 \leq k \leq n$): $e_k \leq w_k$

Nenhum trecho da rede transporta mais energia do que sua capacidade de transmissão permite.

2.4.4 Conservação de fluxo na saída das hidrelétricas

Para cada hidrelétrica i ($1 \leq i \leq h$):
$$\sum_{(i,j) \in A} e_{i,j} = F_i \times V_i.$$

Toda a energia produzida pela hidrelétrica i deve ser escoada pelos arcos que partem dela; a soma da energia nesses arcos é igual à sua produção. É essa restrição que liga as variáveis de vazão às variáveis de fluxo na rede, sem ela, o modelo poderia transmitir energia sem produzi-la, ou produzir energia que nunca entra na rede.

2.4.5 Atendimento das centrais

Para cada central j ($1 \leq j \leq l$):
$$\sum_{(i,k) \in A} e_{i,k} - \sum_{(k,j) \in A} e_{k+h,j} = D_k$$

O primeiro somatório é a energia que chega à central k (arcos cujo destino é a central k); o segundo é a energia que a própria central k repassa a outras centrais (arcos cuja origem é o vértice global $k + h$, correspondente à central k). A diferença entre o que chega e o que sai é a energia efetivamente disponível para atender a demanda D_k .

Optou-se por igualdade, e não por desigualdade ($\geq$), no balanço de cada central: como todos os custos são não negativos, produzir ou transmitir mais energia do que o necessário nunca reduz o custo total, de modo que a solução ótima nunca teria motivo para gerar excedente mesmo que isso fosse permitido. A igualdade apenas torna essa exigência explícita no modelo e evita ambiguidade sobre o que aconteceria com um eventual excedente.

2.4.6 Não negatividade

$V_i \geq 0$ ($1 \leq i \leq h$),

$e_{ij} \geq 0$ ($1 \leq k \leq n$)

2.5 Por que o modelo resolve o problema

Qualquer solução viável do modelo corresponde a uma operação válida da rede: as restrições de capacidade de produção e de vazão do rio respeitam os limites físicos de cada hidrelétrica, as restrições de arco respeitam a infraestrutura de transmissão, a conservação de fluxo garante que toda energia produzida entra na rede, e o balanço de cada central garante que sua demanda é atendida. Reciprocamente, qualquer operação válida da rede pode ser representada no modelo escolhendo V_i igual à vazão realmente utilizada e e_{ij} igual à energia realmente transmitida em cada arco, essa escolha satisfaz todas as restrições e reproduz exatamente o mesmo custo. Há, portanto, uma correspondência entre operações válidas da rede e soluções viáveis do modelo, e minimizar Z sobre essas soluções encontra a operação de custo mínimo.

3. Implementação

O programa foi escrito em C99 e organizado em cinco arquivos-fonte além dos cabeçalhos de dados: *main.c* contém a leitura da entrada e o ponto de entrada do programa; *modelagem.c/modelagem.h* concentram toda a geração do programa linear; e *arco.h*, *central.h* e *hidreletrica.h* definem as estruturas de dados da instância.

3.1 Estruturas de dados

Cada hidrelétrica é representada por uma struct *Hidre*, com os campos capacidade (M_i), eficiência (F_i) e custo (C_i). Cada central é uma struct *Central* com um único campo, demanda (D_i). Cada arco é uma struct *Arco* com quatro campos: origem, destino, capacidade, custo, origem e destino guardando, respectivamente, o índice global do vértice de origem e o índice (local, entre as centrais) do vértice de destino, como descrito na Seção 2.1.

3.2 Leitura da entrada (*ler_dados*, em *main.c*)

A função *ler_dados* recebe os vetores já alocados de hidrelétricas e centrais e um ponteiro para o vetor de arcos, inicialmente nulo. A leitura segue exatamente a ordem do formato de entrada:

- lê, com *scanf*, os três campos (M_i , F_i , C_i) de cada uma das h hidrelétricas, na ordem em que aparecem na entrada;
- lê a demanda D_i de cada uma das l centrais;
- para cada um dos $h + l$ vértices da rede (as h hidrelétricas seguidas das l centrais), lê o número n de vizinhos de saída e, em seguida, os n arcos correspondentes.

Como o número de arcos de cada vértice não é conhecido de antemão, o vetor de arcos é realocado dinamicamente a cada bloco, com *realloc*(*arcos, (*arcos_num) * sizeof(Arco)), crescendo conforme os blocos são lidos. Ao ler os arcos do vértice i (0-indexado no laço, portanto vértice global $i+1$), cada novo arco recebe origem = $i + 1$; o destino, a capacidade e o custo do arco são lidos diretamente do arquivo de entrada, na ordem t , w , c .

Vale notar a diferença de convenção entre os dois campos: origem é sempre o índice global do vértice (1 a $h + l$), calculado internamente pelo próprio laço de leitura; destino é o valor t lido do arquivo, que o enunciado define como o índice da central de destino (1 a l) e não um índice global. É por isso que, mais adiante, o código compara *arcos[j].destino* diretamente com o índice de uma central ($i + 1$), mas soma h ao índice de uma central quando precisa comparar com o campo origem de um arco que sai dela (*hidros_num* + $i + 1$).

3.3 Geração da função objetivo (*funcao_obj*)

A função imprime a linha "min : ..." em duas etapas. Primeiro percorre o vetor de hidrelétricas, imprimindo um termo $\text{custo} * V_i$ para cada uma; depois percorre o vetor de arcos, imprimindo um termo $\text{custo} * e_{\{origem\}_{\{destino\}}}$ para cada um, encerrando a expressão com ";" após o último termo. A concatenação dos termos com "+" e o tratamento especial do último elemento (para não deixar um "+" sobrando no final da linha) são o que exige que o laço sobre os arcos trate o último índice separadamente.

3.4 Geração das restrições (*restricoes*)

A função percorre os mesmos vetores em quatro blocos, na ordem descrita a seguir, cada um correspondendo diretamente a uma das restrições da Seção 2.4:

- Para cada hidrelétrica i : imprime a restrição de capacidade ($F_i * V_i \leq M_i$), a de vazão do rio ($V_i \leq R$), a de não negatividade ($V_i \geq 0$) e, por fim, percorre todo o vetor de arcos somando, com um "+", os termos $e_{\{origem\}_{\{destino\}}}$ de todo arco cuja origem seja a própria hidrelétrica i , formando a equação de conservação de fluxo, fechada com " $= F_i * V_i$ ".
- Para cada arco k : imprime a restrição de capacidade ($e_{\{origem\}_{\{destino\}}} \leq w_{\{k\}}$) e a de não negatividade ($e_{\{origem\}_{\{destino\}}} \geq 0$).

- Para cada central i : percorre o vetor de arcos duas vezes. Na primeira passagem, soma com "+" a energia de todo arco cujo destino seja a própria central (energia que chega); na segunda, subtrai com "-" a energia de todo arco cuja origem seja o vértice global da central ($hidros_num + i + 1$, energia que ela repassa a outras centrais). O resultado é fechado com " $= D_i$ ", formando a equação de atendimento de demanda.

Essa estrutura possui quatro blocos de laços independentes, cada um dedicado a um tipo de restrição mantém a correspondência entre o código e o modelo matemático praticamente linha a linha, o que facilita conferir se a implementação está de fato gerando as restrições descritas na Seção 2.

3.5 Convenção de nomes de variáveis

As variáveis do modelo recebem nomes textuais que o próprio `lp_solve` interpreta como variáveis distintas, sem necessidade de uma seção de declaração: $V\{i\}$ para a vazão da hidrelétrica i ($V1, V2, \dots$) e $e\{k\}_{\{d\}}$ para a energia no arco k , em que d é o índice da central de destino daquele arco (por exemplo, $e1_1$ é o primeiro arco, com destino na central 1). Como o formato de entrada do `lp_solve` reconhece uma variável na primeira vez em que seu nome aparece em qualquer restrição ou na função objetivo[1], essa convenção de nomes é suficiente para que o resolvidor monte corretamente o sistema, sem que o programa precise emitir nenhuma seção adicional de declaração de variáveis.

3.6 Compilação

O makefile fornecido compila todos os arquivos `.c` do diretório `codigos` com `gcc -Wall -g -std=c99` e gera o executável `energia` ao se digitar apenas `make`, como exigido pelo enunciado. Os alvos `clean` e `purge` removem, respectivamente, os arquivos objeto (`.o`) e o próprio executável.

O programa apenas constrói a instância do programa linear; ele não resolve o sistema. A resolução, encontrar os valores ótimos de V_i e $e_{\square}$, ou detectar que a instância é inviável é inteiramente delegada ao `lp_solve`, a quem a saída do programa é fornecida como entrada.

4. Exemplo de uso

4.1 Para a instância de exemplo do enunciado ($h = 2, l = 2, R = 10$), a sequência de comandos é:

```
make
./energia < exemplos/viavel1.txt > viavel1.lp
lp_solve viavel1.lp
```

O arquivo `viavel.lp` gerado pelo programa é, na íntegra:

```
min : 2*V1 + 1*V2 + 1*e1_1 + 2*e2_2 + 1*e3_2;
20*V1 <= 100;
10*V2 <= 50;
V1 <= 10;
V2 <= 10;
e1_1 <= 200;
e2_2 <= 100;
e3_2 <= 100;
e1_1 = 20*V1;
```

```

e2_2 = 10*V2;
e1_1 - e3_2 = 50;
e2_2 + e3_2 = 80;
V1 >= 0;
V2 >= 0;
e1_1 >= 0;
e2_2 >= 0;
e3_2 >= 0;

```

Resolvendo esse arquivo, o lp_solve retorna o valor ótimo:

```
Value of objective function: 223.00000000
```

Actual values of the variables:

V1	4
V2	5
e1_1	80
e2_2	50
e3_2	30

Ou seja: a hidrelétrica 1 produz 80 (com vazão 4), a hidrelétrica 2 produz 50 (com vazão 5), 80 unidades fluem da hidrelétrica 1 para a central 1, 50 da hidrelétrica 2 para a central 2, e 30 são repassadas da central 1 para a central 2. O custo total é

$$2(4) + 1(5) + 1(80) + 2(50) + 1(30) = 223,$$

valor que coincide exatamente com a solução ótima apresentada no enunciado do trabalho, o que confirma a corretude do modelo e da implementação para essa instância. Além disso, as demandas são atendidas com igualdade exata: $80 - 30 = 50$ (central 1) e $50 + 30 = 80$ (central 2).

4.2 Para uma instância inviável, foi utilizada a instância D_alto.txt, composta por duas hidrelétricas e duas centrais elétricas, com vazão máxima do rio igual a 10. Essa instância possui duas hidrelétricas. A primeira apresenta capacidade máxima de produção igual a 100, eficiência 20 e custo de produção 2, enquanto a segunda possui capacidade máxima de 50, eficiência 10 e custo de produção 1. As duas centrais possuem demanda igual a 80.

```

make
./energia < exemplos/D_alto.txt > inviavel_D.lp
lp_solve inviavel_D.lp

```

O arquivo inviavel_D.lp gerado pelo programa é, na íntegra:

```

min : 2*V1 + 1*V2 + 1*e1_1 + 2*e2_2 + 1*e3_2;
20*V1 <= 100;
10*V2 <= 50;
V1 <= 10;
V2 <= 10;
e1_1 <= 200;
e2_2 <= 100;

```

```

e3_2 <= 100;
e1_1 = 20*V1;
e2_2 = 10*V2;
e1_1 - e3_2 = 80;
e2_2 + e3_2 = 80;
V1 >= 0;
V2 >= 0;
e1_1 >= 0;
e2_2 >= 0;
e3_2 >= 0;

```

Resolvendo esse arquivo, o lp_solve retorna que o problema é inviável:

```

This problem is infeasible

```

Uma instância pode ser inviável quando a demanda das centrais excede a capacidade de produção das hidrelétricas ou quando as capacidades da rede de transmissão impedem que energia suficiente chegue às centrais. Nesses casos, não existe uma combinação de vazões e fluxos que satisfaça simultaneamente todas as restrições do modelo.

5. Nota sobre inviabilidade

Uma instância é inviável quando não existe nenhuma atribuição de vazões e de fluxos que satisfaça simultaneamente todas as restrições, em particular, quando a rede não consegue entregar a alguma central energia suficiente para cobrir sua demanda com igualdade exata.

Como praticamente todas as restrições do modelo (capacidade de produção, vazão do rio, capacidade dos arcos) são do tipo $\leq$, e apenas o balanço de demanda é uma igualdade, a inviabilidade está sempre associada a uma limitação de capacidade em algum ponto do sistema. Duas condições são particularmente relevantes:

- Insuficiência de produção agregada: se a soma das produções máximas possíveis de todas as hidrelétricas, $\sum_i \min(M_i, F_i \cdot R)$, for menor que a soma de todas as demandas $\sum_{k=1}^l D_k$, a instância é inviável, não há energia suficiente em lugar nenhum da rede, independentemente de como ela seja distribuída.
- Corte de capacidade insuficiente: mesmo com produção agregada suficiente, uma instância é inviável se existir um corte na rede, um conjunto de arcos e de capacidades de produção cuja remoção desconecta as hidrelétricas de uma ou mais centrais, cuja capacidade total seja menor que a demanda do lado das centrais desse corte. Esse é essencialmente o teorema do corte mínimo–fluxo máximo aplicado à rede de transmissão [2]: uma central isolada por arcos de baixa capacidade pode ser inviável mesmo que a produção total do sistema seja abundante.

O uso de igualdade (em vez de desigualdade) no balanço de demanda, discutido na Seção 2.4.5, não introduz uma fonte adicional de inviabilidade: como o modelo é contínuo (não há variáveis inteiras) e todas as demais restrições permanecem do tipo $\leq$, qualquer solução em que uma central receba mais energia do que sua demanda pode ser ajustada — reduzindo a vazão da hidrelétrica correspondente e o fluxo no arco excedente, até que a igualdade seja atingida exatamente, sem violar nenhuma outra restrição.

Em qualquer um dos dois casos, a implementação não precisa detectar a inviabilidade: ela apenas traduz a instância para o formato do *lp_solve*, e é o próprio resolvidor quem, ao não encontrar uma solução viável, reporta a instância como inviável.

6. Conclusão

O problema de produção, transmissão e demanda de energia foi modelado como um programa linear cuja estrutura corresponde a um problema de fluxo em rede com custos, múltiplas fontes (as hidrelétricas) e múltiplos pontos de demanda (as centrais). A implementação em C lê uma instância no formato especificado no enunciado e gera diretamente o arquivo de entrada do *lp_solve*, delegando a esse resolvidor a tarefa de encontrar a solução ótima ou de identificar a inviabilidade da instância. A validação com a instância de exemplo do enunciado reproduziu exatamente o resultado ótimo esperado custo total 223, confirmando a corretude da modelagem e da implementação.

7. Referências

- [1] LP_SOLVE. *lp_solve reference guide: lp_format*. Disponível em: <http://lpsolve.sourceforge.net/5.5/lp-format.htm>. Acesso em: 14 set. 2026.
- [2] MATOUŠEK, J.; GÄRTNER, B. *Understanding and Using Linear Programming*. Springer, 2007, p. 148 (teorema do fluxo máximo–corte mínimo). Disponível em: <http://link.springer.com/book/10.1007%2F978-3-540-30717-4>.
- [3] UNIVERSIDADE FEDERAL DO PARANÁ. Departamento de Informática. *CI1238/INFO7070 — Otimização: Primeiro Trabalho Prático*. 27 ago. 2026. Disponível em: <https://www.inf.ufpr.br/andre/Disciplinas/CI1238-2026-2/trabalho1.pdf>.